

MAD & PWA Lab Journal

Experiment No.	08
Experiment Title.	To code and register a service worker, and complete the install and activation process for a new service worker for the E-commerce PWA
Roll No.	02
Name	Vanshika Ambwani
Class	D15A
Subject	MAD & PWA Lab
Lab Outcome	LO5: Design and Develop a responsive User Interface by applying PWA Design techniques
Grade:	

Aim: To code and register a service worker, and complete the install and activation process for a new service worker for the E-commerce PWA.

Theory:

Service Worker

Service Worker is a script that works on browser background without user interaction independently. Also, It resembles a proxy that works on the user side. With this script, you can track network traffic of the page, manage push notifications and develop “offline first” web applications with Cache API.

Things to note about Service Worker:

- A service worker is a programmable network proxy that lets you control how network requests from your page are handled.
- Service workers only run over HTTPS. Because service workers can intercept network requests and modify responses, "man-in-the-middle" attacks could be very bad.
- The service worker becomes idle when not in use and restarts when it's next needed. You cannot rely on a global state persisting between events. If there is information that you need to persist and reuse across restarts, you can use IndexedDB databases.

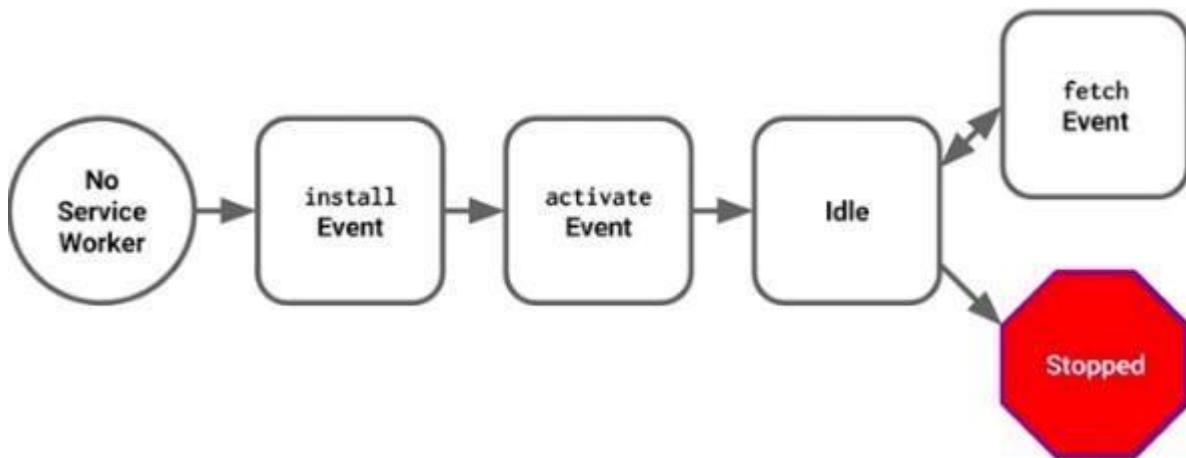
What can we do with Service Workers?

- You can dominate **Network Traffic**
You can manage all network traffic of the page and do any manipulations. For example, when the page requests a CSS file, you can send plain text as a response or when the page requests an HTML file, you can send a png file as a response. You can also send a true response too.
- You can **Cache**
You can cache any request/response pair with Service Worker and Cache API and you can access these offline content anytime.
- You can manage **Push Notifications**
You can manage push notifications with Service Worker and show any information message to the user.
- You can **Continue**
Although Internet connection is broken, you can start any process with Background Sync of Service Worker.

What can't we do with Service Workers?

- You can't access the **Window**
You can't access the window, therefore, You can't manipulate DOM elements. But, you can communicate to the window through post Message and manage processes that you want.
- You can't work it on **80 Port**
Service Worker just can work on HTTPS protocol. But you can work on localhost during development.

Service Worker Cycle



A service worker goes through three steps in its life cycle:

- Registration
- Installation
- Activation

Registration

To install a service worker, you need to register it in your main JavaScript code. Registration tells the browser where your service worker is located, and to start installing it in the background. Let's look at an example:

main.js

```
if ('serviceWorker' in navigator) {  
  navigator.serviceWorker.register('/service-worker.js')  
    .then(function(registration) { console.log('Registration successful,  
      scope is:', registration.scope);  
    })  
    .catch(function(error) { console.log('Service worker  
      registration failed, error:', error); });  
}
```

This code starts by checking for browser support by examining **navigator.serviceWorker**. The service worker is then registered with `navigator.serviceWorker.register`, which returns a promise that resolves when the service worker has been successfully registered. The scope of the service worker is then logged with `registration.scope`. If the service worker is already installed, `navigator.serviceWorker.register` returns the registration object of the currently active service worker.

The scope of the service worker determines which files the service worker controls, in other words, from which path the service worker will intercept requests. The default scope is the location of the service worker file, and

extends to all directories below. So if service-worker.js is located in the root directory, the service worker will control requests from all files at this domain.

You can also set an arbitrary scope by passing in an additional parameter when registering. For example:

main.js

```
navigator.serviceWorker.register('/service-worker.js', {  
  scope: '/app/'  
});
```

In this case we are setting the scope of the service worker to /app/, which means the service worker will control requests from pages like /app/, /app/lower/ and /app/lower/lower, but not from pages like /app or /, which are higher.

If you want the service worker to control higher pages e.g. /app (without the trailing slash) you can indeed change the scope option, but you'll also need to set the Service-Worker-Allowed HTTP Header in your server config for the request serving the service worker script.

main.js

```
navigator.serviceWorker.register('/app/service-worker.js', {  
  scope: '/app'  
});
```

Installation

Once the browser registers a service worker, installation can be attempted. This occurs if the service worker is considered to be new by the browser, either because the site currently doesn't have a registered service worker, or because there is a byte difference between the new service worker and the previously installed one.

A service worker installation triggers an install event in the installing service worker. We can include an install event listener in the service worker to perform some task when the service worker installs. For instance, during the install, service workers can precache parts of a web app so that it loads instantly the next time a user opens it (see caching the application shell). So, after that first load, you're going to benefit from instant repeat loads and your time to interactivity is going to be even better in those cases. An example of an installation event listener looks like this:

service-worker.js

```
// Listen for install event, set callback  
self.addEventListener('install', function(event) {  
// Perform some task
```

```
});
```

Activation

Once a service worker has successfully installed, it transitions into the activation stage. If there are any open pages controlled by the previous service worker, the new service worker enters a waiting state. The new service worker only activates when there are no longer any pages loaded that are still using the old service worker. This ensures that only one version of the service worker is running at any given time.

When the new service worker activates, an activate event is triggered in the activating service worker. This event listener is a good place to clean up outdated caches (see the Offline Cookbook for an example).

service-worker.js

```
self.addEventListener('activate', function(event) { //  
    Perform some task  
});
```

Once activated, the service worker controls all pages that load within its scope, and starts listening for events from those pages. However, pages in your app that were loaded before the service worker activation will not be under service worker control. The new service worker will only take over when you close and reopen your app, or if the service worker calls **clients.claim()**. Until then, requests from this page will not be intercepted by the new service worker. This is intentional as a way to ensure consistency in your site.

Code

index.html

Vanshika Ambwani (02), Gunjan Chandnani (06), Akruti Dabas (11)

```
<? index.html X JS service-worker.js README.md index.txt manifest.json script.js
index.html > html > body#top > main > article > section#collection.section.collection > div.container > ul.collection-list > li > div.collection-card.has-before.hovers:shine
1 <!DOCTYPE html>
2 <html lang="en">
3
4 <head>
5   <link rel="manifest" href="manifest.json">
6
7   <meta charset="UTF-8">
8   <meta http-equiv="X-UA-Compatible" content="IE=edge">
9   <meta name="viewport" content="width=device-width, initial-scale=1.0">
10  <title>Glowing - Reveal The Beauty of Skin</title>
11
12
13  <!--
14  | - favicon
15  -->
16  <link rel="shortcut icon" href="./favicon.svg" type="image/svg+xml">
17
18  <!--
19  | - custom css link
20  -->
21  <link rel="stylesheet" href="./assets/css/style.css">
22
23  <!--
24  | - google font link
25  -->
26  <link rel="preconnect" href="https://fonts.googleapis.com">
27  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
28  <link href="https://fonts.googleapis.com/css2?family=Urbanist:wght@400;500;600;700;800&display=swap" rel="stylesheet">
29
30  <!--
31  | - preload images
32  -->
33  <link rel="preload" as="image" href="./assets/images/logo.png">
34  <link rel="preload" as="image" href="./assets/images/hero-banner-1.jpg">
35  <link rel="preload" as="image" href="./assets/images/hero-banner-2.jpg">
36  <link rel="preload" as="image" href="./assets/images/hero-banner-3.jpg">
37
```

service-worker.js

```
self.addEventListener("install", (event) => {
  console.log("Service Worker Installed");
});

self.addEventListener("fetch", (event) => {
  event.respondWith(fetch(event.request));
});
```

manifest.json

```
{
  "name": "Shopping App",
  "short_name": "MyApp",
  "description": "An online shopping platform with a wide range of products.",
  "start_url": "/",
  "scope": "/",
```

```
"display": "standalone",
"background_color": "#ffffff",
"theme_color": "#ff6600",
"icons": [
  {
    "src": "assets/images/banner-1.png",
    "sizes": "192x192",
    "type": "image/png",
    "purpose": "any maskable"
  },
  {
    "src": "assets/images/banner-2.png",
    "sizes": "512x512",
    "type": "image/png",
    "purpose": "any maskable"
  }
],
"shortcuts": [
  {
    "name": "Shop Now",
    "short_name": "Shop",
    "description": "Go directly to shopping",
    "url": "/index.html",
    "icons": [
      {
        "src": "assets/images/shortcut-icon.png",
        "sizes": "96x96",
        "type": "image/png"
      }
    ]
  }
]
```

